Data Mining

Week 9

Submitter - Himanshu Singh

Best Model Selection and Hyperparameter Tuning

In this exercise, you will work with the Loan_Train.csv dataset which can be downloaded from this link: Loan Approval Data Set.

- Import the dataset and ensure that it loaded properly.
- Prepare the data for modeling by performing the following steps:
  - Drop the column "Load_ID."
  - Drop any rows with missing data.
  - Convert the categorical features into dummy variables.
- Split the data into a training and test set, where the "Loan_Status" column is the target.
- Create a pipeline with a min-max scaler and a KNN classifier (see section 15.3 in the Machine Learning with Python Cookbook).
- Fit a default KNN classifier to the data with this pipeline. Report the model accuracy on the test set. Note: Fitting a pipeline model works just like fitting a regular model.
- Create a search space for your KNN classifier where your "n_neighbors" parameter varies from 1 to 10. (see section 15.3 in the Machine Learning with Python Cookbook).
- Fit a grid search with your pipeline, search space, and 5-fold cross-validation to find the best value for the "n_neighbors" parameter.
- Find the accuracy of the grid search best model on the test set. Note: It is possible that this will not be an improvement over the default model, but likely it will be.
- Now, repeat steps 6 and 7 with the same pipeline, but expand your search space to include logistic regression and random forest models with the hyperparameter values in section 12.3 of the Machine Learning with Python Cookbook.
- What are the best model and hyperparameters found in the grid search? Find the accuracy of this model on the test set.
- Summarize your results.

In [12]:
```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load the dataset
df = pd.read_csv('data/Loan_Train.csv')

# Inspect the data
print(df.head())
print(df.info())
print(df.isnull().sum())
```

```
    Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001002   Male      No          0      Graduate            No
1  LP001003   Male     Yes          1      Graduate            No
2  LP001005   Male     Yes          0      Graduate           Yes
3  LP001006   Male     Yes          0  Not Graduate            No
4  LP001008   Male      No          0      Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5849                0.0         NaN             360.0
1             4583             1508.0       128.0             360.0
2             3000                0.0        66.0             360.0
3             2583             2358.0       120.0             360.0
4             6000                0.0       141.0             360.0

   Credit_History Property_Area Loan_Status
0             1.0         Urban           Y
1             1.0         Rural           N
2             1.0         Urban           Y
3             1.0         Urban           Y
4             1.0         Urban           Y
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            614 non-null    object
 1   Gender             601 non-null    object
 2   Married            611 non-null    object
 3   Dependents         599 non-null    object
 4   Education          614 non-null    object
 5   Self_Employed      582 non-null    object
 6   ApplicantIncome    614 non-null    int64
 7   CoapplicantIncome  614 non-null    float64
 8   LoanAmount         592 non-null    float64
 9   Loan_Amount_Term   600 non-null    float64
 10  Credit_History     564 non-null    float64
 11  Property_Area      614 non-null    object
 12  Loan_Status        614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
None
Loan_ID               0
Gender               13
Married               3
Dependents           15
Education             0
Self_Employed        32
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           22
Loan_Amount_Term     14
Credit_History       50
Property_Area         0
Loan_Status           0
dtype: int64
```

In [ ]:

```python
# Step 2a and 2b: Drop Loan_ID and missing rows
data = df.drop(columns=['Loan_ID']).dropna()

# Step 2c: Convert target Loan_Status to numeric (Y=1, N=0)
```

```python
data['Loan_Status'] = data['Loan_Status'].map({'Y': 1, 'N': 0})

# Step 2c: Convert categorical features into dummy variables
# Identify categorical columns (excluding target)
cat_features = data.select_dtypes(include=['object']).columns
data_dummies = pd.get_dummies(data, columns=cat_features, drop_first=True)

# print(data_dummies.head())
# print(cat_features.tolist())


# Step 3: Split the data into features and target, then into training and test sets
X = data_dummies.drop(columns=['Loan_Status'])
y = data_dummies['Loan_Status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Create a pipeline with a min-max scaler and a KNN classifier with default parameters
pipe = Pipeline([
    ('scaler', MinMaxScaler()),
    ('classifier', KNeighborsClassifier())
])

# Step 5: Fit a default KNN classifier and find its accuracy on the test set
pipe.fit(X_train, y_train)
y_pred_default = pipe.predict(X_test)
acc_default = accuracy_score(y_test, y_pred_default)

# Step 6: Create a search space for KNN n_neighbors (1 to 10)
search_space_knn = [{'classifier__n_neighbors': list(range(1, 11))}]

# Step 7: Fit a grid search with 5-fold CV
grid_knn = GridSearchCV(pipe, search_space_knn, cv=5, verbose=0).fit(X_train, y_train)
best_n = grid_knn.best_params_['classifier__n_neighbors']

# Step 8: Find accuracy of best KNN on test set using the best n_neighbors found in the grid search
y_pred_best_knn = grid_knn.predict(X_test)
acc_best_knn = accuracy_score(y_test, y_pred_best_knn)

# Step 9: Expand search space to include Logistic Regression and Random Forest
# Using parameters typically found in section 12.3 of the cookbook
search_space_multi = [
    {
        'classifier': [KNeighborsClassifier()],
        'classifier__n_neighbors': list(range(1, 11))
    },
    {
        'classifier': [LogisticRegression(solver='liblinear')],
        'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100]
    },
    {
        'classifier': [RandomForestClassifier()],
        'classifier__n_estimators': [10, 100, 1000],
        'classifier__max_features': [1, 2, 3]
    }
]

grid_multi = GridSearchCV(pipe, search_space_multi, cv=5, verbose=0).fit(X_train, y_train)

# Results of multi-model search
```

```python
best_model = grid_multi.best_estimator_.get_params()['classifier']
best_params = grid_multi.best_params_
y_pred_best_multi = grid_multi.predict(X_test)
acc_best_multi = accuracy_score(y_test, y_pred_best_multi)

print(f"Default KNN Accuracy: {acc_default:.4f}")
print(f"Best KNN n_neighbors: {best_n}")
print(f"Best KNN Accuracy: {acc_best_knn:.4f}")
print(f"Best Model Overall: {best_model}")
print(f"Best Params Overall: {best_params}")
print(f"Best Model Test Accuracy: {acc_best_multi:.4f}")

# KNN Optimization: Fitting a default KNeighborsClassifier yielded an accuracy of 78.12%.
# After conducting a grid search for the optimal number of neighbors (n_neighbors between 1 and 10),
# the best value was found to be 3, which improved the test accuracy to 79.17%.

# Model Selection: We expanded the search space to include Logistic Regression and Random Forest models.
# The grid search identified Logistic Regression with a regularization strength of C=10 as the superior model.
# Final Best Model: The Logistic Regression model achieved the highest accuracy of 82.29% on the test set,
# outperforming both the optimized KNN and the Random Forest configurations tested.

# The analysis demonstrates that while tuning the K parameter improves the performance of a KNN model,
# Logistic Regression proved to be the more effective algorithm for this specific classification task.
# The final model is capable of predicting loan approval status with approximately 82.3% accuracy,
# providing a solid baseline for credit risk assessment in this dataset.
```

```
Default KNN Accuracy: 0.7812
Best KNN n_neighbors: 3
Best KNN Accuracy: 0.7917
Best Model Overall: LogisticRegression(C=10, solver='liblinear')
Best Params Overall: {'classifier': LogisticRegression(solver='liblinear'), 'classifier__C': 10}
Best Model Test Accuracy: 0.8229
```